

5.项目突发情况处理

小组权限改动配置

失败后再次操作

结合你当前现象、版本错乱、SSH 认证失败、多用户共享配置的背景，给你**分步修复方案**，先回退到指定版本、修复SSH问题，再做权限&使用规则约束，避免普通用户乱改版本。

一、问题总结

1. 你指定启动 `K8s v1.27.16`，但环境被改动、启动异常，集群卡死；
2. 中途 `Ctrl+C` 中断启动，容器/配置不完整，引发 **SSH 握手失败**；
3. 多用户共用 Minikube，`admin4` 重启时篡改版本/启动参数，导致集群版本错乱、配置冲突；
4. 你用 `root + --force` 启动 Docker 驱动的 minikube，本身也容易叠加权限问题。

二、完整修复步骤（按顺序执行）

步骤1：彻底销毁异常集群 + 清理残留配置

先清空损坏的实例、缓存、共享配置，杜绝旧配置干扰：

代码块

```
1  # 1. 删除当前异常集群
2  minikube delete
3
4  # 2. 清理 root 本地缓存
5  rm -rf /root/.minikube /root/.kube
6
7  # 3. 清理你们公共共享目录（关键，版本错乱根源）
8  rm -rf /opt/teamwork/minikube-config /opt/teamwork/kubeconfig
```

步骤2：root 身份严格按指定参数重启集群（锁定版本 v1.27.16）

后续只允许 `root` 执行 `minikube start/stop/delete`，普通用户禁止操作

代码块

```
1 minikube start \  
2   --driver=docker \  
3   --cpus=3 \  
4   --memory=6144 \  
5   --kubernetes-version=v1.27.16 \  
6   --image-mirror-country=cn \  
7   --base-image=gcr.io/k8s-minikube/kicbase:v0.0.46
```

- 不要加 `--force`（root 用 docker 驱动加 `--force` 极易破坏容器内SSH密钥）；
- 等待全程启动完成，**不要中途 Ctrl+C 中断**。

步骤3：验证集群恢复（必须无报错）

代码块

```
1 minikube status  
2 kubectl version
```

正常输出：

- host / kubelet / apiserver 全部 `Running`
- Kubernetes Server 版本为 `v1.27.16`
- 不再出现 `ssh: unable to authenticate`

步骤4：重新生成公共共享配置（给 4 个用户共用）

重建共享目录、复制配置、替换路径，同时**严格收紧密钥权限**（防止再次SSH报错）：

代码块

```
1 # 1. 重建公共目录  
2 mkdir -p /opt/teamwork/minikube-config  
3  
4 # 2. 复制 minikube 完整配置到公共目录  
5 cp -r /root/.minikube/* /opt/teamwork/minikube-config/  
6
```

```
7 # 3. 复制 kubeconfig
8 cp /root/.kube/config /opt/teamwork/kubeconfig
9
10 # 4. 批量替换路径 /root/.minikube → 公共目录
11 sed -i 's|/root/.minikube|/opt/teamwork/minikube-config|g'
    /opt/teamwork/kubeconfig
12
13 # 5. 归属统一为 teamgroup 组
14 chown -R root:teamgroup /opt/teamwork/minikube-config /opt/teamwork/kubeconfig
15
16 # 6. 精细化权限 (核心: 修复SSH认证问题)
17 chmod 775 /opt/teamwork/minikube-config
18 # 机器密钥目录强制 700
19 chmod 700 /opt/teamwork/minikube-config/machines
20 # 所有私钥 600
21 find /opt/teamwork/minikube-config -name "id_rsa" -exec chmod 600 {} \;
22 find /opt/teamwork/minikube-config -name "*.key" -exec chmod 600 {} \;
23 # 公钥/证书 644
24 find /opt/teamwork/minikube-config -name "*.pub" -exec chmod 644 {} \;
25 find /opt/teamwork/minikube-config -name "*.crt" -exec chmod 644 {} \;
26 # kubeconfig 只读
27 chmod 644 /opt/teamwork/kubeconfig
```

步骤5：逐个配置普通用户环境变量（admin4 + 其他3位用户）

以 `admin4` 为例，其余用户操作一致：

代码块

```
1 # 编辑用户环境变量
2 vi /home/admin4/.bashrc
```

文末追加：

代码块

```
1 # 共用公共 minikube & k8s 配置
2 export MINIKUBE_HOME=/opt/teamwork/minikube-config
3 export KUBECONFIG=/opt/teamwork/kubeconfig
```

保存退出，加载生效并验证：

```
1 su - admin4
2 source .bashrc
3 minikube status
4 kubectl get nodes
```

能正常查询、无SSH报错、版本为 `v1.27.16` 即为正常。

三、限制普通用户：禁止 admin4 等修改/重启 Minikube（根治版本乱改）

方案1：使用 sudo 黑名单/指令限制（推荐）

1. 编辑 sudo 配置

代码块

```
1 visudo
```

2. 禁止组内用户执行 minikube 启停/删除命令

在文件末尾添加：

代码块

```
1 # 禁止 teamgroup 组执行 minikube start/stop/delete
2 %teamgroup ALL=(ALL) !/usr/bin/minikube start, !/usr/bin/minikube stop,
   !/usr/bin/minikube delete
```

作用：

- 组内用户哪怕用 `sudo`，也无法执行集群启停、删除，**从命令层面锁死版本篡改**。

方案2：补充口头+文档规则（小组强制遵守）

1. **唯一运维账号**：仅 `root` 有权执行 `minikube start / stop / delete`、修改启动参数、切换K8s版本；
2. 普通用户权限范围：仅允许 `kubectl` 查询、执行项目部署脚本、查看日志；
3. 禁止普通用户私自执行 `minikube` 带参数的启动命令；

4. 所有启动参数、K8s 版本 **统一固定为 v1.27.16**，不再变更。

四、后续业务恢复（部署你的微服务）

代码块

```
1 # 切回 root / 任意普通用户均可
2 cd /opt/teamwork/ceshi2026
3 ./deploy/scripts/deploy-all.sh
4 kubectl get pods
5 ./deploy/scripts/port-forward.sh
```

浏览器访问 <http://47.94.7.122:8080> 验证业务正常。

六、故障复现规避总结

1. 不再使用 `--force` 配合 root + docker 驱动启动 minikube；
2. 严禁中途 `Ctrl+C` 中断集群启动；
3. 共享配置目录**永远精细化权限**，不再执行 `chmod -R 777 /` `chmod -R g+s`；
4. 命令层面限制普通用户启停集群，锁定 K8s 版本 `v1.27.16`；
5. 如需调整集群参数/版本，全员沟通后由 root 统一操作。

1、回到镜像包目录，把分包全部载入宿主机 docker

bash

运行

代码块

```
1 cd /opt/microservice-project
2 docker load -i part1.tar
3 docker load -i part2.tar
4 docker load -i part3.tar
```

```
5 docker load -i part4.tar
6 docker load -i busybox.tar
7 docker load -i redis.tar
8 # 校验宿主机docker有全套镜像docker images | grep google-samples
```

2、打通宿主机 docker → minikube 内部 docker（一步同步镜像）

bash

运行

代码块

```
1 # 切换当前终端docker上下文到minikube内部eval $(minikube docker-env)# 现在docker操作
  全是minikube虚拟机里的docker images
2 # 这里能直接看到所有gcr.io/redis镜像，不用load、不
```

方案 1：把宿主机镜像打包推送到 minikube（稳妥）

1、退出 minikube docker 上下文

新开一个终端操作，或者执行取消环境变量：

bash

运行

代码块

```
1 unset DOCKER_TLS_VERIFY DOCKER_HOST DOCKER_CERT_PATH DOCKER_API_VERSION
```

此时 `docker images` 又能看到你全套 google-samples 镜像。

2、逐个推送镜像到 minikube 内部

bash

运行

代码块

```
1 minikube image load part1.tar
2 minikube image load part2.tar
3 minikube image load part3.tar
4 minikube image load redis.tar
5 minikube image load busybox.tar
```

你本地已经有完整 docker 标准 tar 分包，直接 load 进 minikube，比 ctr 导出 OCI 稳定太多。

加载完再切上下文验证：

bash

运行

代码块

```
1 eval $(minikube docker-env)docker images | grep google-samples
```

修改每个yaml文件的image